Réseaux de Neurones Multicouches (Multi Layer Perceptron: MLP)

Rose Bandolo

9 mai 2025

Table de matiere

1. Architecture des Réseaux de Neurones à Couches Multiples (MLP)
2. feedforward (propagation avant)
3. Backpropagation
4. Fonctions d'activations
5. The Vanishing Gradient Problem
6. Overfitting Vs underfitting
7. Regularisation

Architecture des Réseaux de Neurones à Couches Multiples (MLP)

- Le perceptron multicouche est un réseau neuronal artificiel. Il s'agit généralement d'un problème d'apprentissage supervisé non linéaire.

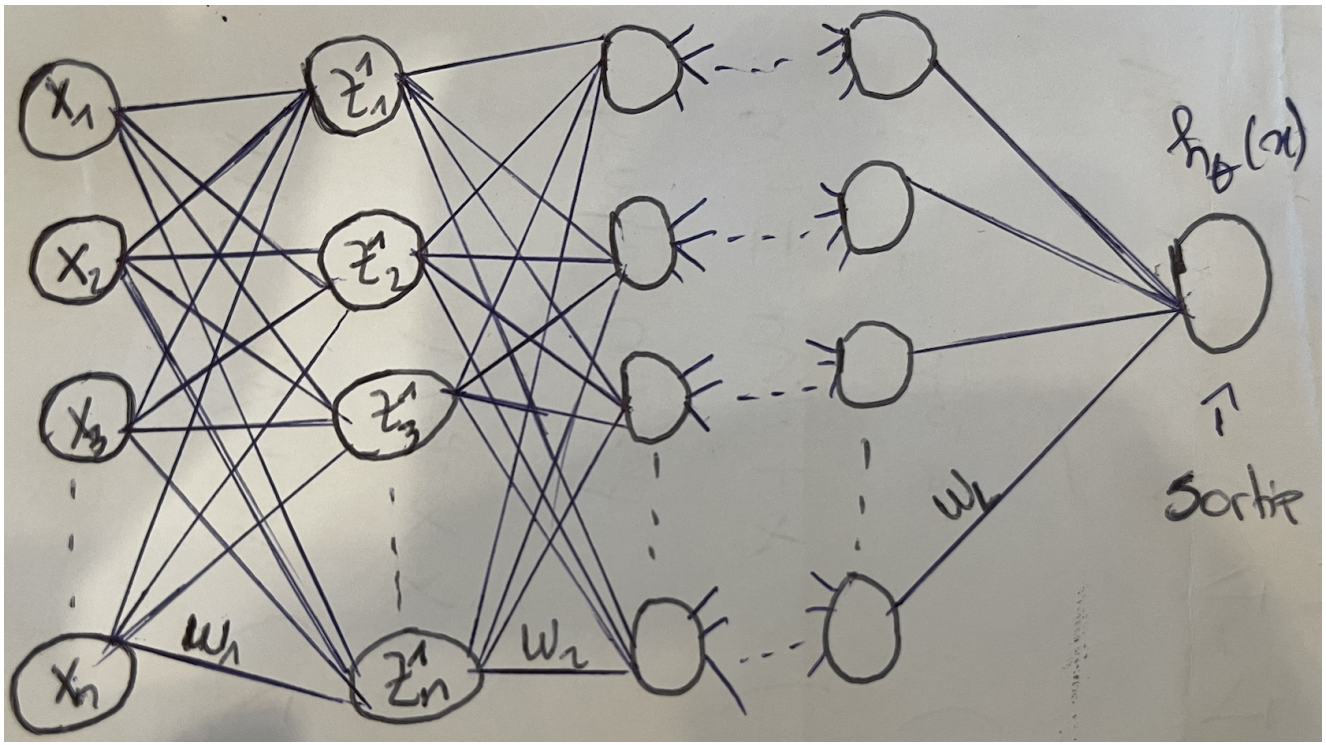


Figure 1 – Structure du perceptron multicouche

Réseaux de Neurones à Couches Multiples (MLP)

D'après la figure 1

$$z_i^1 = \sigma(W_1^{(i)\,T} x + b_1) \qquad i = 1, ..., L \tag{1}$$

$$z_1 = \sigma(W_1^T x + \vec{b}_1), \quad \vec{b}_1 = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \qquad W_1 = \begin{bmatrix} w_1^{(1)} \\ w_2^{(1)} \\ \vdots \\ w_n^{(1)} \end{bmatrix} \tag{2}$$

Le vecteur de sorties pour la deuxième couche cachée est donné par

$$z_2 = \sigma(W_2^T z_1 + \vec{b}_2) = \sigma(W_2^T \sigma(W_1^T x + \vec{b}_1) + \vec{b}_2) \tag{3}$$

Pour la dernière couche, $z_L = h_\theta(x)$ où

$$h_\theta(x) = \sigma(W_L^T \sigma(W_{L-1}^T \sigma(W_{L-2}^T \cdots \sigma(W_2^T \sigma(W_1^T x + b_1) + b_2) + \cdots) + b_{L-1}) \tag{4}$$

Réseaux de Neurones à Couches Multiples (MLP)

C'est une composition de fonctions composées de transformations affines et d'une fonction non linéaire. Les paramètres sont

$$\theta = \{W_L, W_{L-1}, \cdots, W_1, b_L, b_{L-1}, \cdots, b_1\} \tag{5}$$

Notez qu'un perceptron multicouche entièrement connectée est également appelée réseau neuronal à couches denses, comme le montre la figure 2. Pour compter le nombre de couches d'un réseau neuronal, nous ne comptons pas la couche d'entrée. Par exemple, la figure 2 est un réseau neuronal à 2 couches.

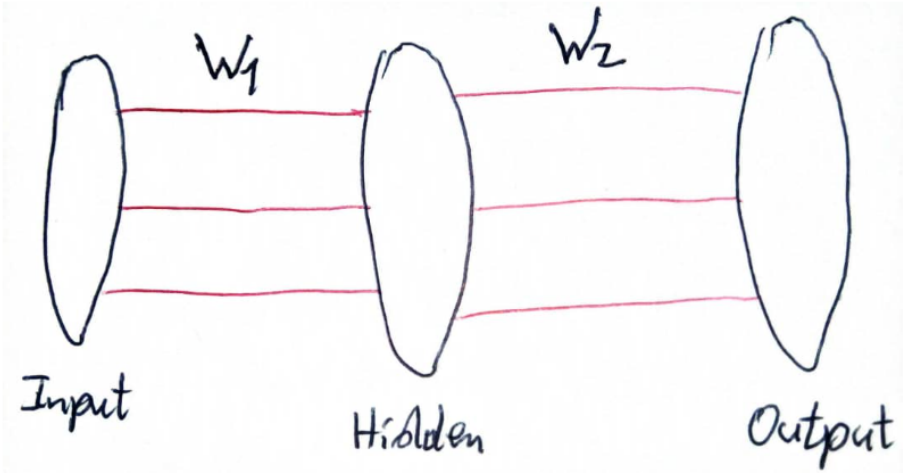


Figure 2 – Réseau de neurones à 2 couches

feedforward (propagation avant)

- **Couche d'Entrée :** Les données d'entrée sont introduites dans la première couche du réseau.
- **Propagation :** Les données sont propagées à travers les couches cachées du réseau. Chaque neurone dans une couche cachée calcule une combinaison linéaire de ses entrées pondérées par des poids, puis applique une fonction d'activation (comme ReLU, sigmoïde, ou tanh) à la somme pondérée pour produire une sortie.
- **Couche de Sortie :** Les activations de la dernière couche cachée sont propagées à travers la couche de sortie, qui génère la sortie finale du réseau.
- **Prédiction :** La sortie de la couche de sortie est interprétée comme la prédiction du réseau pour la tâche donnée, qu'il s'agisse de classification, de régression ou d'une autre tâche.

Entraînement d'un Réseau de Neurone Multicouches : Le Backpropagation

- Le **Backpropagation** est la phase d'apprentissage dans les réseaux de neurones.
- Il s'agit de rétropropager l'erreur de prédiction à travers le réseau pour ajuster les poids.
- On compare la prédiction avec les vraies étiquettes pour calculer l'erreur (fonction de coût).
- L'erreur est propagée en sens inverse, de la couche de sortie vers la couche d'entrée.
- Les gradients par rapport aux poids sont calculés pour chaque couche grâce à la règle de la chaîne.
- Les poids sont ajustés pour minimiser l'erreur à l'aide d'algorithmes d'optimisation tels que la descente de gradient.

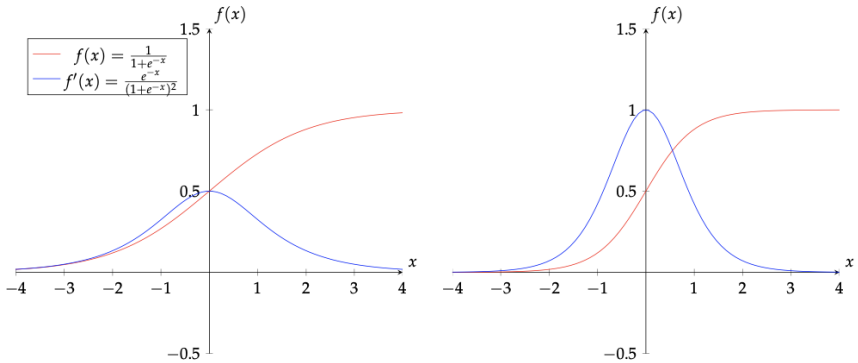
Fonctionnement du Backpropagation (Suite)

- Le processus de Backpropagation est itératif, et il est répété sur plusieurs cycles d'apprentissage.
- Les poids sont mis à jour à chaque cycle pour améliorer la précision des prédictions.
- Le Backpropagation permet au réseau de "savoir" comment ajuster ses poids pour minimiser l'erreur.
- Avec suffisamment de cycles d'apprentissage, le réseau apprendra à faire des prédictions précises.

Fonction d'activation

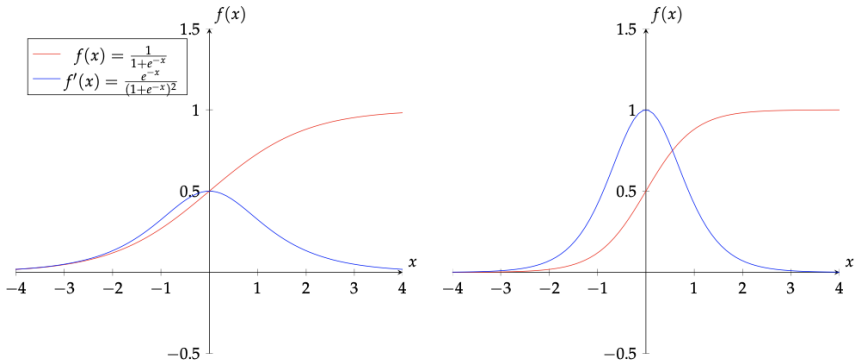
- La fonction d'activation d'un réseau permet d'introduire de la non-linéarité dans la fonction $y = f(x)$.
- C'est elle qui permet de transformer les données afin qu'elles soient linéairement séparables.
- Plusieurs fonctions d'activation sont possibles, parmi lesquelles :La fonction sigmoïde,tangente hyperbolique, ReLU, softplus.

Fonctions sigmoïde et tangente hyperbolique



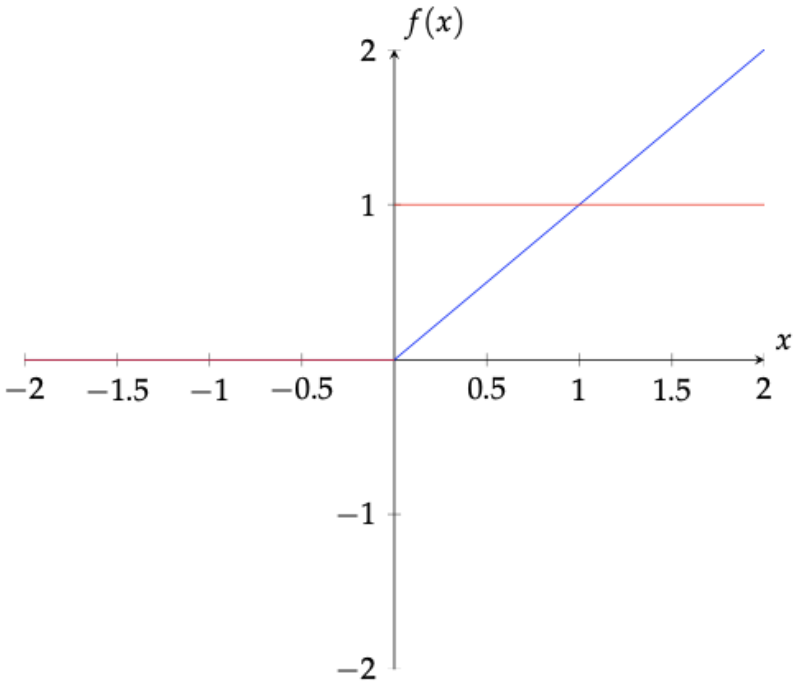
- Elles ont l'avantage d'être dérivables, ce qui permet de calculer le gradient de la fonction d'erreur.
- Elles ont l'inconvénient de posséder des dérivées qui prennent des valeurs très faibles sur la majeure partie de leur domaine. Lors du calcul du gradient de la fonction d'erreur d'un réseau profond, on réalise des multiplications en chaîne de la fonction dérivée.

Fonctions sigmoïde et tangente hyperbolique



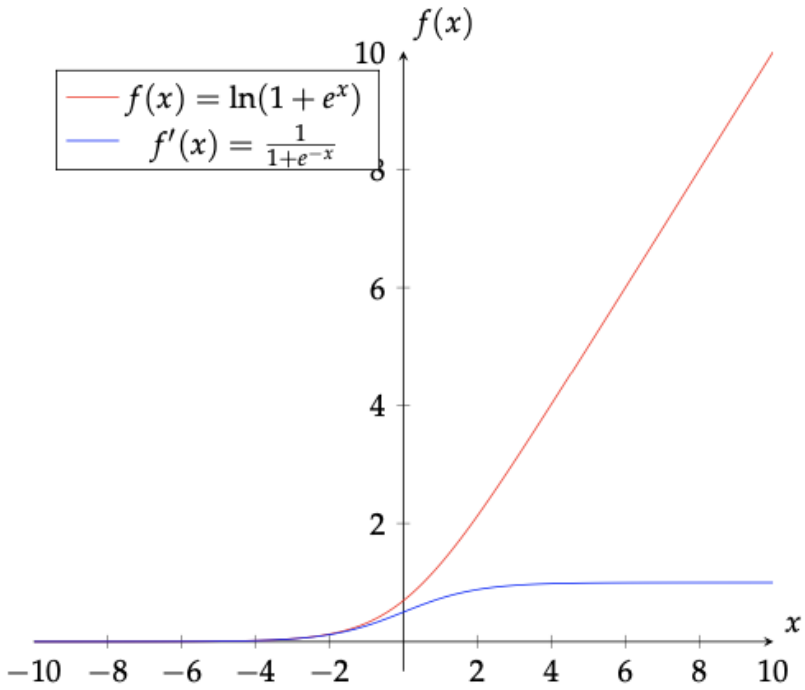
- Le gradient a tendance à prendre des valeurs de plus en plus petites, phénomène connu sous le nom de disparition du gradient ou vanishing gradient .
- Lors de l'apprentissage, les couches inférieures (proches de l'entrée) sont modifiées très lentements, voire pas du tout.

La fonction ReLU



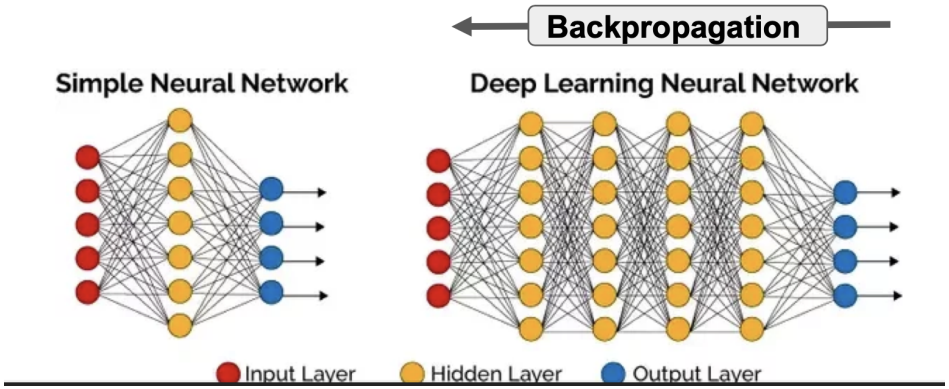
- Elle échappe au problème de la disparition du gradient. Mais elle n'est pas dérivable en 0.
- Dans la pratique cela n'est pas très problématique car la valeur 0 est très rarement atteinte. Mais ce n'est pas très joli

La fonction SoftPlus



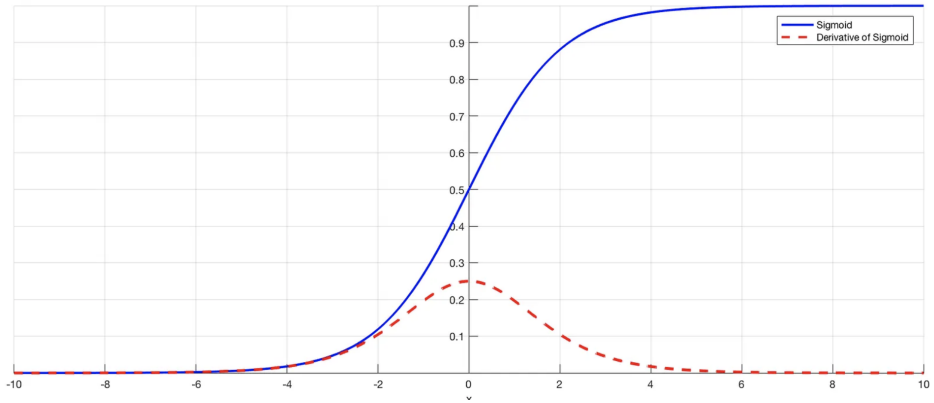
- Version dérivable de ReLU
- La dérivée de SoftPlus est la fonction sigmoïde
- Peu utilisée en pratique

Problème du Gradient qui Disparaît (The Vanishing Gradient Problem)



- se produit lorsque les gradients des poids deviennent très petits à mesure que l'on se propage vers l'arrière du réseau lors de l'entraînement.
- Cela peut rendre l'apprentissage difficile, en particulier pour les réseaux profonds.

Problème du Gradient qui Disparaît (The Vanishing Gradient Problem)



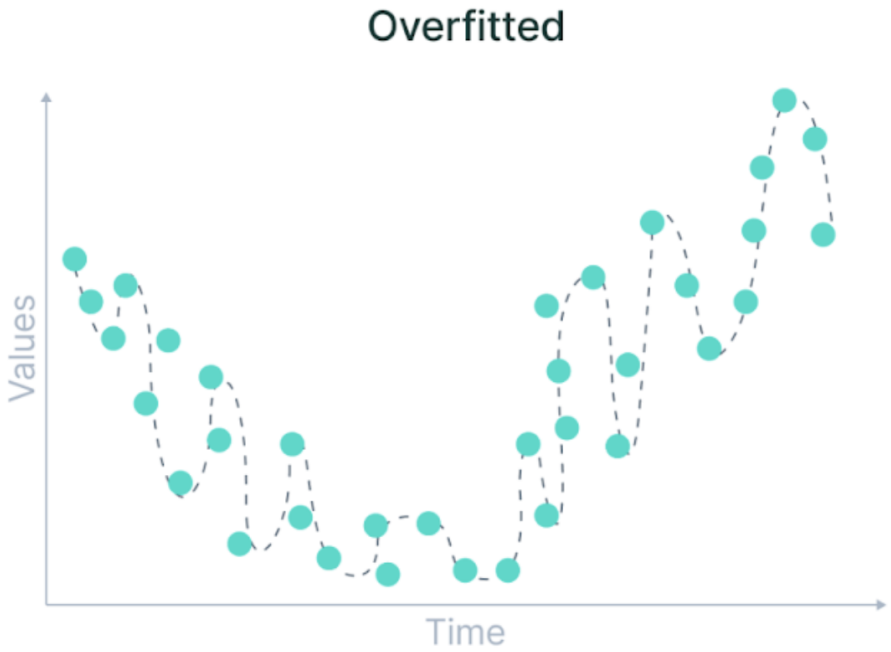
- Le problème du gradient qui disparaît est lié à l'utilisation de fonctions d'activation telles que la sigmoïde ou la tangente hyperbolique (tanh) dans les couches cachées des réseaux de neurones.
- Ces fonctions d'activation ont une caractéristique commune : elles compriment leur plage de sortie dans un intervalle limité (0,1) ,(-1,1).

Problème du Gradient qui Disparaît (The Vanishing Gradient Problem)

Solution :

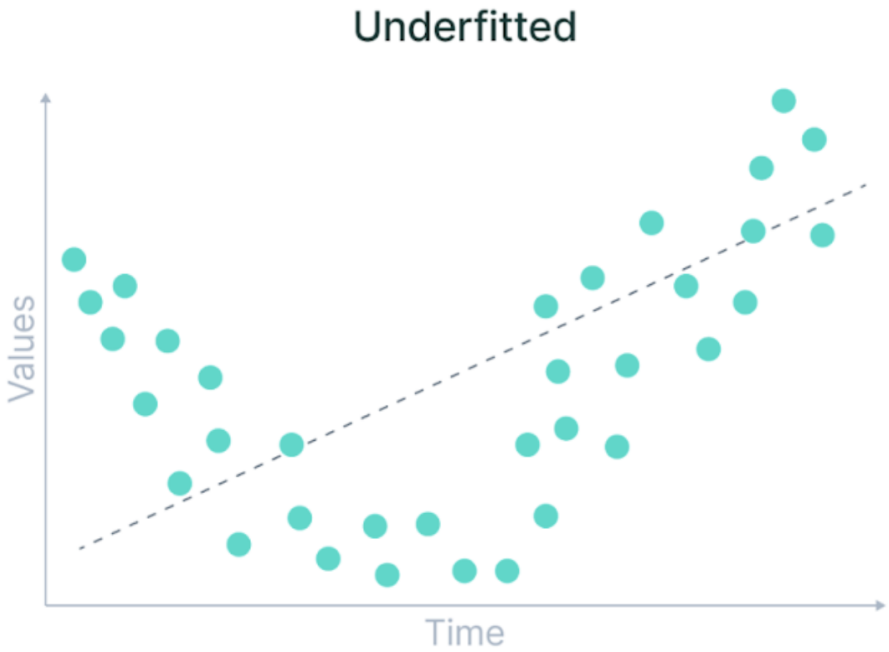
- Utilisation de fonctions d'activation différentes telles que ReLU (Rectified Linear Unit). ses gradients sont constants dans certaines plages, ce qui empêche le gradient de disparaître.
- Batch Normalization : Cette technique consiste à normaliser les activations à chaque couche du réseau, ce qui peut stabiliser l'apprentissage en maintenant les activations à une échelle appropriée.
- Long Short-Term Memory (LSTM) : Dans les réseaux récurrents, les LSTM utilisent une structure de porte qui permet de conserver et de propager efficacement les informations temporelles, réduisant ainsi le problème du gradient qui disparaît.

Overfitting Vs underfitting



- Surajustement (Overfitting) : Le surajustement se produit lorsque le modèle s'est trop adapté aux données d'entraînement, capturant le bruit et les détails au lieu des tendances générales. Cela se traduit par une mauvaise généralisation aux données de test.

Overfitting Vs underfitting



- Sous-ajustement (Underfitting) : Le sous-ajustement se produit lorsque le modèle est trop simple pour capturer les motifs dans les données d'entraînement. Le modèle ne parvient pas à apprendre les relations importantes et ne performe pas bien sur les données de test.

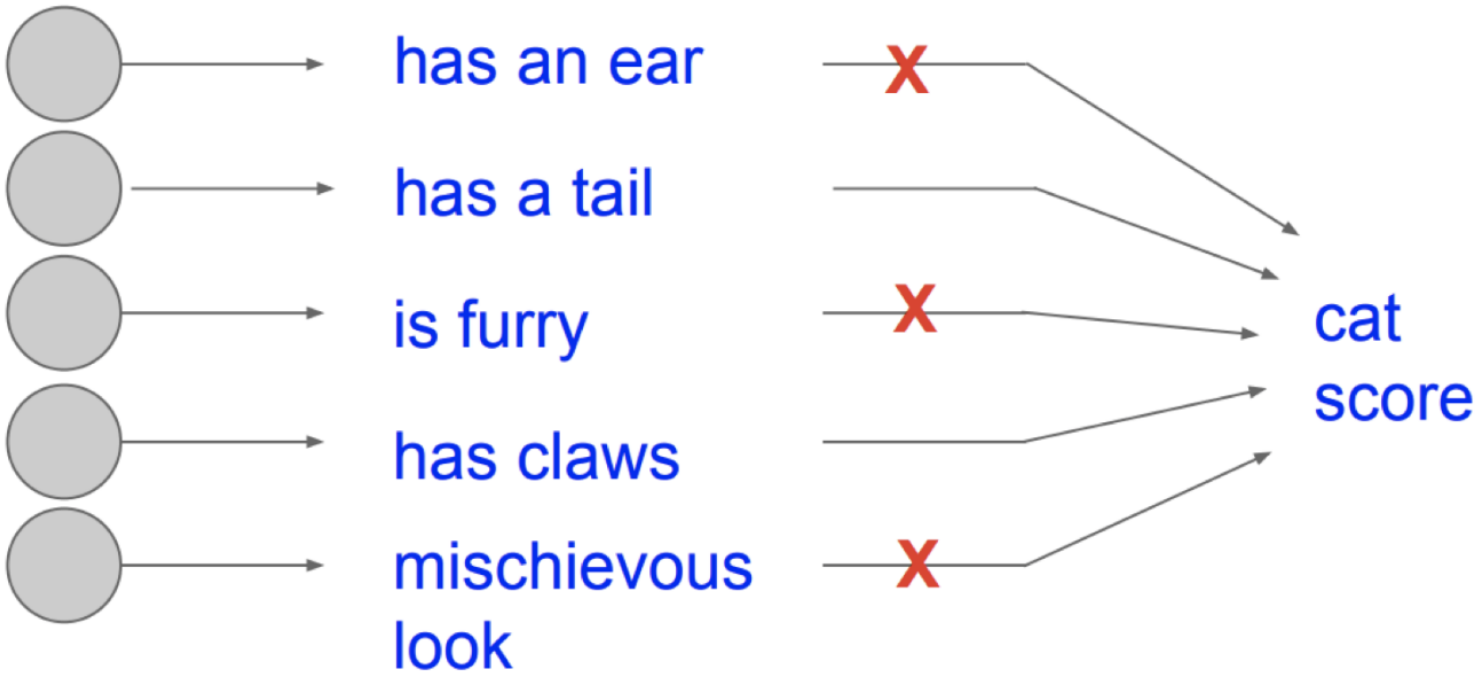
Régularisation : Dropout

- Réseau original/standard : Toutes les caractéristiques sont transmises au classificateur.



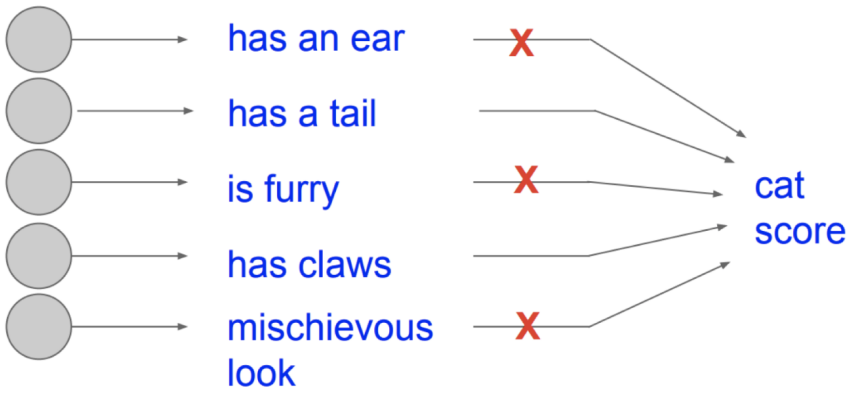
Régularisation : Dropout

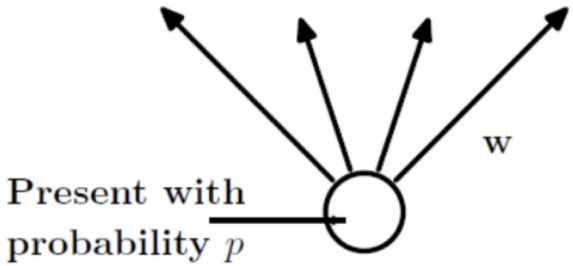
- **Dropout** : Abandonner de manière aléatoire certaines entrées.



Régularisation : Dropout

- Les neurones sont conservés/abandonnés avec une probabilité p

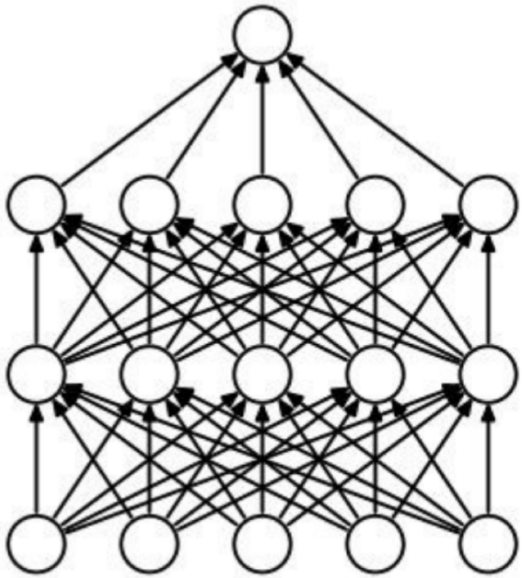




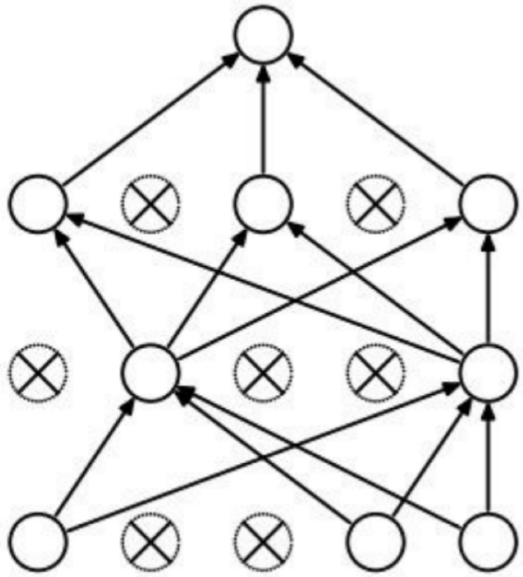
(a) At training time

Régularisation : Dropout

- Les neurones sont conservés/abandonnés avec une probabilité p



(a) Standard Neural Net



(b) After applying dropout.

Régularisation : Dropout

- Avantages :
  - Certains neurones/entrées sont abandonnés.
  - D'autres neurones sont obligés de compenser, d'apprendre une meilleure représentation générique.
  - Meilleure représentation générique : Empêche le surajustement(overfitting).
- Ajoute du bruit aux gradients :
  - Empêche le surajustement
  - Aide à la généralisation avec du bruit (pour représenter des données inédites à partir d'une distribution similaire).
- Réduire la complexité du modèle :
  - Moins de neurones
  - Empêche le surajustement.

Régularisation :Batch Normalization

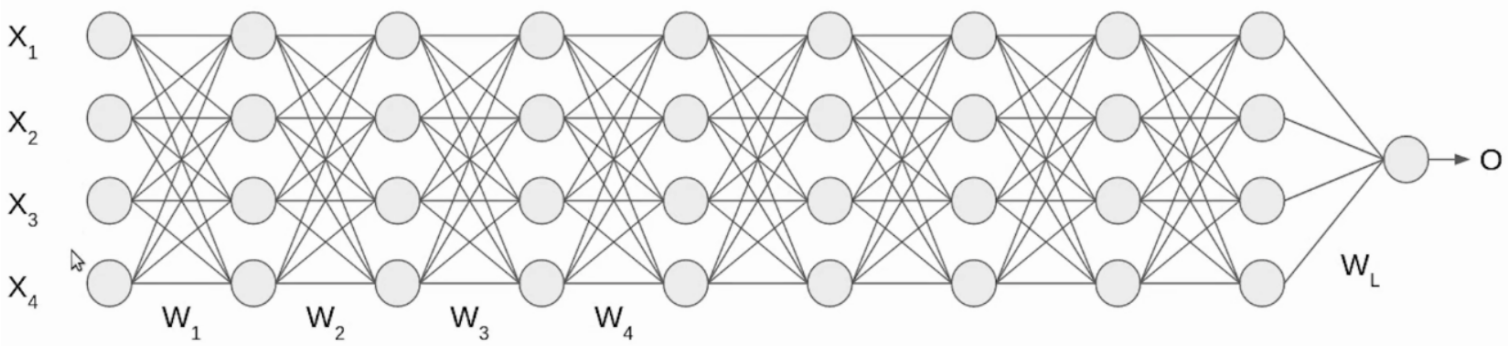
Le Batch Normalization ou Normalisation par lots :

- est un processus visant à rendre les réseaux neuronaux plus rapides et plus stables en ajoutant des couches supplémentaires dans un réseau neuronal profond.
- La nouvelle couche effectue les opérations de normalisation et de standardisation sur l'entrée d'une couche provenant d'une couche précédente.

Régularisation :Batch Normalization

Pourquoi la normalisation par lots ? ?

- Au départ, nos entrées X1, X2, X3, X4 sont sous forme normalisée car elles proviennent de l'étape de prétraitement.

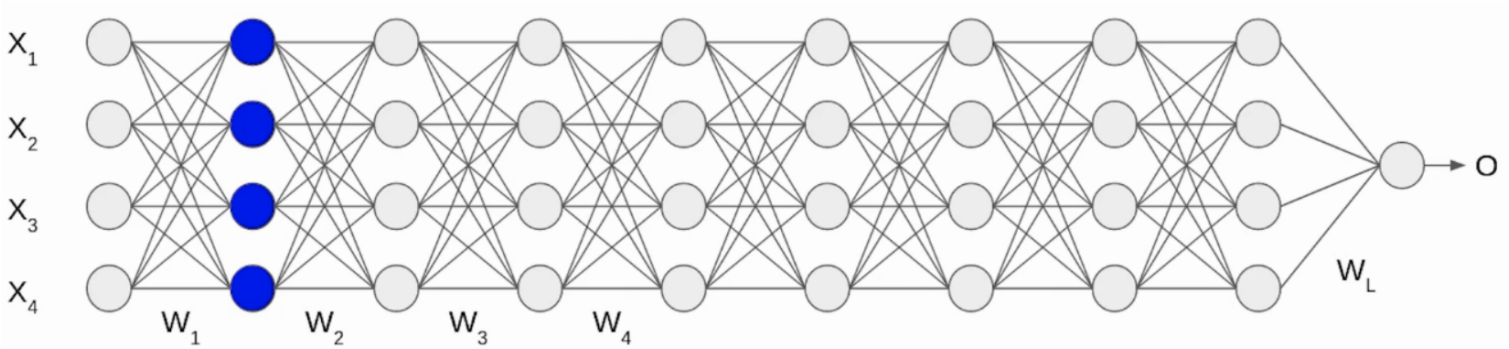


L= Nombres de couches, biais= 0, fonction d'activation : Sigmoïde

Régularisation :Batch Normalization

Pourquoi la normalisation par lots ? ?

- Lorsque l'entrée traverse la première couche, elle se transforme, etant donné qu'une fonction sigmoïde est appliquée au produit de points de l'entrée X et de la matrice de poids W.



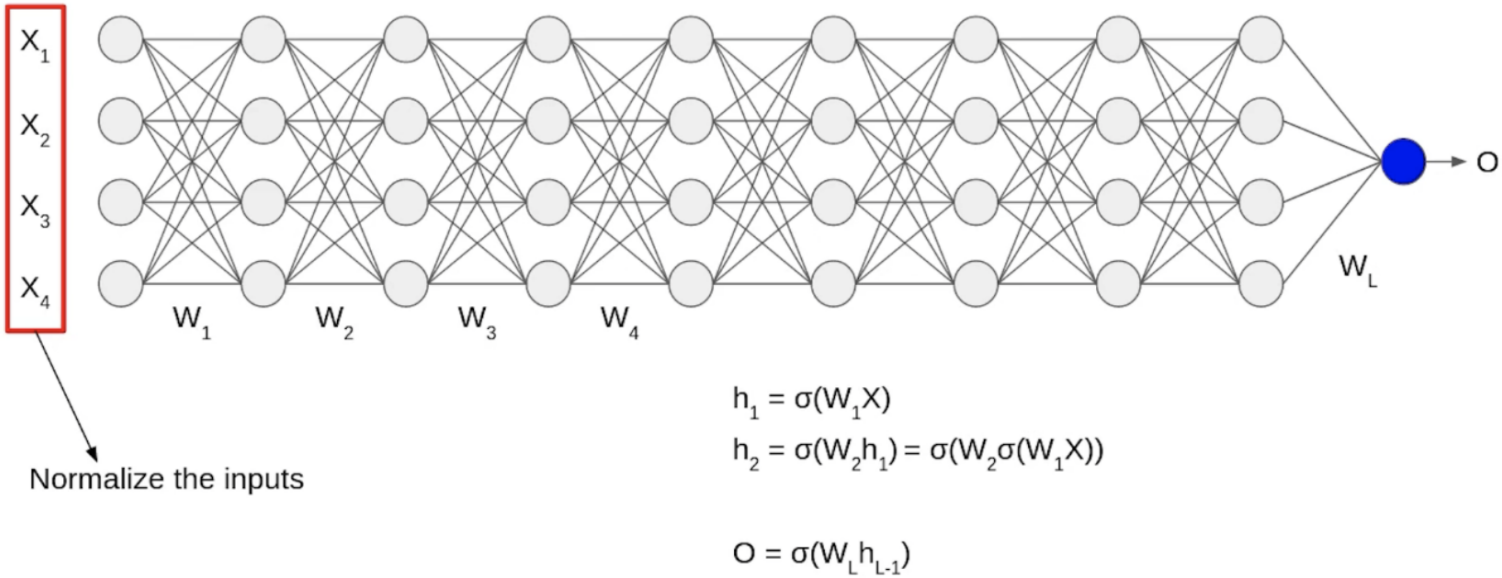
fonction d'activation : Sigmoïde

$$h_1 = \sigma(W_1 X)$$

Régularisation :Batch Normalization

Pourquoi la normalisation par lots ? ?

- De la même manière, cette transformation s'applique à la deuxième couche jusqu'à la dernière couche L, comme le montre l'image suivante.



Bien que notre entrée X ait été normalisée avec le temps, la sortie ne sera plus à la même échelle. Lorsque les données passent par plusieurs couches du réseau neuronal et que L fonctions d'activation sont appliquées, les données subissent un décalage de covariance .

Régularisation :Batch Normalization

Comment fonctionne la normalisation par lots : Normalisation de l'entrée

- Dans cette étape, nous avons notre entrée de lot de la couche h, nous devons d'abord calculer la moyenne de cette activation cachée.

$$\mu = \frac{1}{m}\sum h_i$$

Ici, m est le nombre de neurones de la couche h.

- Une fois la moyenne obtenue, l'étape suivante consiste à calculer l'écart-type des activations cachées.

$$\sigma = \left[\frac{1}{m}\sum (h_i - \mu)^2\right]^{1/2}$$

Régularisation :Batch Normalization

Comment fonctionne la normalisation par lots : Normalisation de l'entrée

- De plus, comme nous disposons de la moyenne et de l'écart-type, nous allons normaliser les activations cachées à l'aide de ces valeurs.
- Pour ce faire, nous soustrayons la moyenne de chaque entrée et divisons la valeur totale par la somme de l'écart type et du terme de lissage (ϵ).
- Le terme de lissage (ϵ) assure la stabilité numérique de l'opération en arrêtant la division par une valeur nulle.

$$h_{i(nom)} = \frac{h_i - \mu}{\sigma + \epsilon}$$

Régularisation :Batch Normalization

Comment fonctionne la normalisation par lots : Remise à l'échelle et Decalage

- Dans la dernière opération, la remise à l'échelle et le décalage de l'entrée ont lieu.
- C'est là qu'interviennent deux composantes, γ(gamma) et β (bêta).
- Ces paramètres sont utilisés pour la remise à l'échelle (γ) et le décalage (β) du vecteur contenant les valeurs des opérations précédentes.

$$h_i = \gamma h_{i(nom)} + \beta$$